CSE 816 – SOFTWARE PRODUCTION ENGINEERING · MINI-PROJECT (TERM I, 2026-27)

End-to-End CI/CD Pipeline for a Calculator REST API

Student: chandrahas121 · **Repository:** https://github.com/chandrahas121/spe_miniProject · **Docker Hub:** https://hub.docker.com/r/chandrahas121/calculator

1. Overview

A small calculator REST API written in plain Java (the JDK's built-in HTTP server, built with Maven, tested with JUnit 5) is wrapped in a fully automated pipeline. A push to GitHub starts a Jenkins pipeline that builds the code, runs the tests, builds a Docker image, pushes it to Docker Hub and deploys it with Docker Compose behind an nginx reverse proxy, without any manual step. The application is deliberately simple; the effort went into the automation around it.

2. Architecture

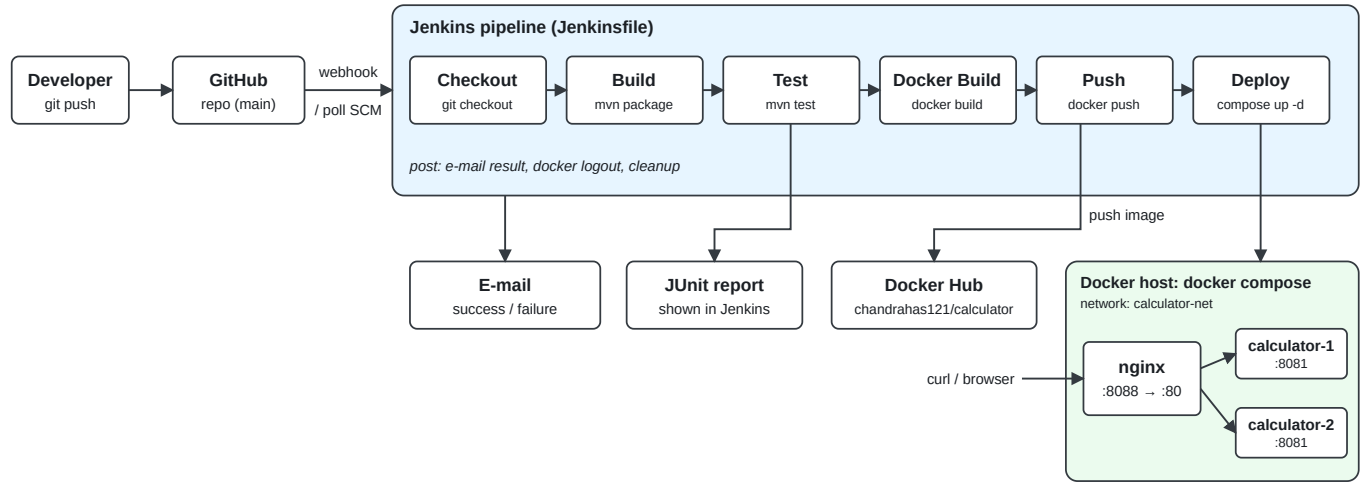


Figure 1. A push to GitHub starts the Jenkins pipeline (GitHub webhook, with Poll SCM every 5 minutes as fallback). The same image that was pushed to Docker Hub is deployed on the Docker host, where nginx spreads requests over two calculator replicas.

3. Application

Endpoint (all GET)	Behaviour
`/api/v1/add` `subtract` `multiply` `divide` `?a=&b=`	JSON reply such as `{"operation":"add","a":2,"b":3,"result":5}`. Numbers are `BigDecimal`, so `0.1 + 0.2` is exactly `0.3`; division is rounded to 16 significant digits.
Bad input	HTTP 400 with a JSON body for a missing or non-numeric parameter, `NaN`, absurd exponents such as `1e999999999`, and division by zero. Unknown paths give 404, non-GET methods give 405.
`/actuator/health`	`{"status":"UP"}`, used by the Docker/Compose health checks and by the Deploy stage.

Tests: 28 JUnit 5 tests run by `mvn test`: 14 unit tests of the calculator (every operation, decimals, negatives, both divide-by-zero cases) and 14 integration tests that start the real server on a random port and call it with `java.net.http.HttpClient`. Surefire writes JUnit XML to `target/surefire-reports`, which Jenkins publishes.

Note on the framework: the problem statement mentions Spring Boot. Following the TA's instruction to keep the application a simple Java program, the framework was dropped. The REST endpoints, the `/actuator/health` URL, Maven packaging and JUnit XML reporting are unchanged, and the MockMvc test became an HTTP integration test.

4. Jenkins pipeline stages

Stage	What it does	Notes
Checkout	`checkout scm` pulls the latest commit of `main`.	Triggered by `githubPush()`; `pollSCM` every 5 min as fallback.
Build	`./mvnw -B clean package -DskipTests` produces `target/calculator.jar`.	Maven wrapper, so Jenkins needs no Maven install.
Test	`./mvnw -B test`, then the `junit` step publishes the XML reports.	A failing test fails the build; results are still published.

Docker Build	`docker build` tagged `chandrahas121/calculator:<build number>` and `:latest`.	Multi-stage Dockerfile (section 5).
Push	`docker login` with the Jenkins-managed credential `dockerhub-creds`, then push both tags.	Token never in the repo or the log (masked). The login lives in a per-build `DOCKER_CONFIG` folder that is deleted afterwards.
Deploy	`IMAGE_TAG=<build number> docker compose up -d --no-build`, then polls `http://localhost:8088/actuator/health` until it answers `UP` (60 s limit).	Re-runnable: unchanged containers are left alone. The build fails if the health check never passes.
Post	E-mail on success and on failure, `docker logout`, cleanup.	Mail errors are caught so a notification problem cannot fail a good deploy.

Other options: builds never run concurrently (two deploys would fight over the ports), 20-minute timeout, last 15 builds kept, timestamps in the log.

5. Containerization and deployment

- **Dockerfile** (multi-stage): a `maven:3.9-eclipse-temurin-21` stage builds the JAR (`pom.xml` is copied first so the dependency layer is cached until the pom changes); the runtime stage is `eclipse-temurin:21-jre-alpine` and copies only the JAR. It runs as a non-root user, has an explicit `EXPOSE 8081`, uses the exec-form `ENTRYPOINT` so `docker stop` reaches Java, and a `.dockerignore` keeps the build context small. A single `docker run -p 8081:8081` serves the API.
- **Compose** (`docker-compose.yml`): service `calculator` with 2 replicas, a health check on `/actuator/health`, `restart: unless-stopped` and a `PORT` environment variable; service `nginx` (port 8088) that waits for healthy replicas and balances round-robin (the `X-Upstream` header shows which replica answered); a user-defined network `calculator-net`. One `docker compose up -d` starts everything.
- **Checked behaviour:** a second `up -d` changes nothing; a new image tag recreates only the calculators while nginx keeps serving; a crashed replica is restarted automatically.

6. Git practices

Work was split into feature branches (`feature/calculator-api`, `feature/docker-compose`, `feature/jenkins-pipeline`, `feature/report`), each merged into `main` with a merge commit, using small commits with plain messages. `.gitignore` excludes `target/`, IDE files, `.env` and keys. No credential is in the repository: the Docker Hub token and the notification address exist only in the Jenkins credentials store.

7. Challenges faced and how they were solved

Problem	Solution
Jenkins already occupies port 8080.	App on 8081 inside the container, nginx published on 8088.
Docker pulls failed on the development machine: its Docker config pointed to a missing credential helper (`docker-credential-desktop`).	The pipeline uses its own `DOCKER_CONFIG` folder in the workspace, so it does not depend on any user-level Docker config.
Jenkins could not run `docker` (permission denied).	Added the `jenkins` user to the `docker` group and restarted the service.
Compose derived the project name from the folder name. The Jenkins workspace has a different folder name, so a deploy would have started a second stack fighting for port 8088.	Fixed the project name (`name: calculator-devops`) in the compose file.
nginx resolved the replica addresses only at start-up, so after a redeploy (new container IPs) it kept pointing at dead addresses.	Variable `proxy_pass` with Docker's resolver (`127.0.0.11`, 5 s validity), so addresses are re-resolved while running.
Gmail refuses normal account passwords for SMTP (error 535). Jenkins' "Test configuration" button does not save the settings, so the first build fell back to `localhost:25`.	Use a Google app password (needs 2-step verification) and press Save after the test. The mail step is wrapped in `try/catch` so a mail problem never fails the deploy.

A pipeline cannot be fully tried without running it.	The Jenkinsfile was validated with Jenkins' declarative linter and run once in a disposable Jenkins container with dummy credentials, confirming stage order, credential masking, JUnit publishing and the failure path before the real run.

8. Result

Build #1 of the `calculator-pipeline` job finished with **SUCCESS**: 28 tests passed, the image `chandrahas121/calculator:1` (and `:latest`) is public on Docker Hub, and the stack is running with nginx and both replicas reporting healthy; `curl "localhost:8088/api/v1/add?a=2&b=3"` returns `{"operation":"add","a":2,"b":3,"result":5}`. Screenshots of the green Jenkins run, the e-mail, `docker ps` and sample API responses are submitted separately.